

PROGRAMACIÓN DE MICROCONTROLADORES

REPASO DE C - primera parte

**Carrera de Especialización en Sistemas Embebidos
Facultad de Ingeniería - UBA**

Empezamos con una pregunta

¿Cómo evalúa sus conocimientos de C?

1. <https://www.menti.com/u3ugkt81eb>
2. [Ver resultados](#)

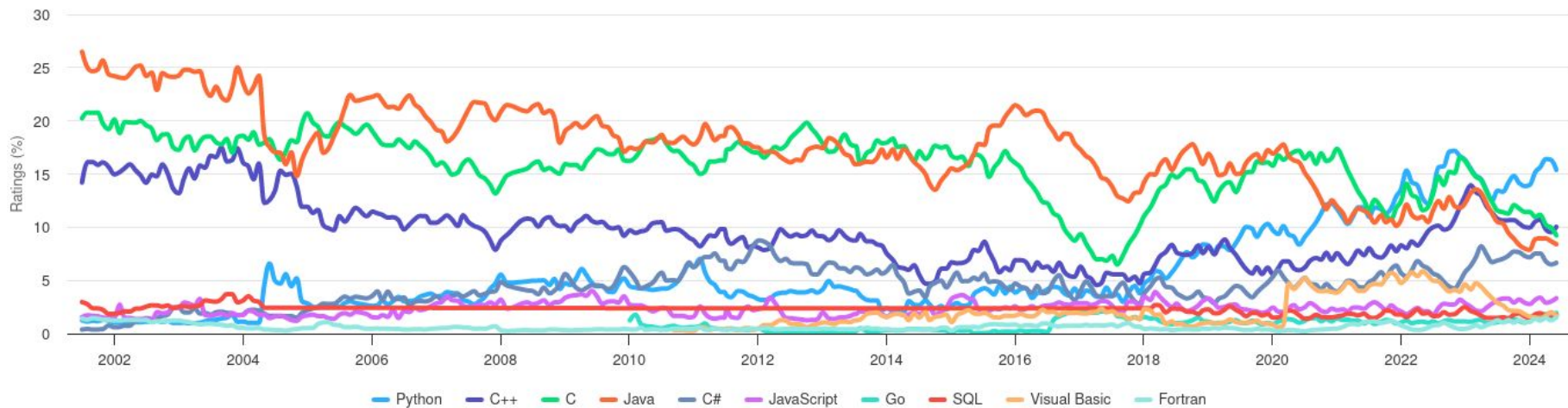


Repaso de C - Motivaciones

- El lenguaje de programación 'C' fue originalmente desarrollado para re-implementar el sistema operativo UNIX por Dennis Ritchie en 1971.
- 'C' no está atado a ningún hardware ni sistema en particular.
- "Middle-level", combina elementos de los lenguajes de alto nivel con funciones de bajo nivel como acceso directo a memoria.
- 'C' retiene la filosofía de que el programador sabe lo que está haciendo.

TIOBE Programming Community Index

Source: www.tiobe.com



<https://www.tiobe.com/tiobe-index/>

Assembly vs C

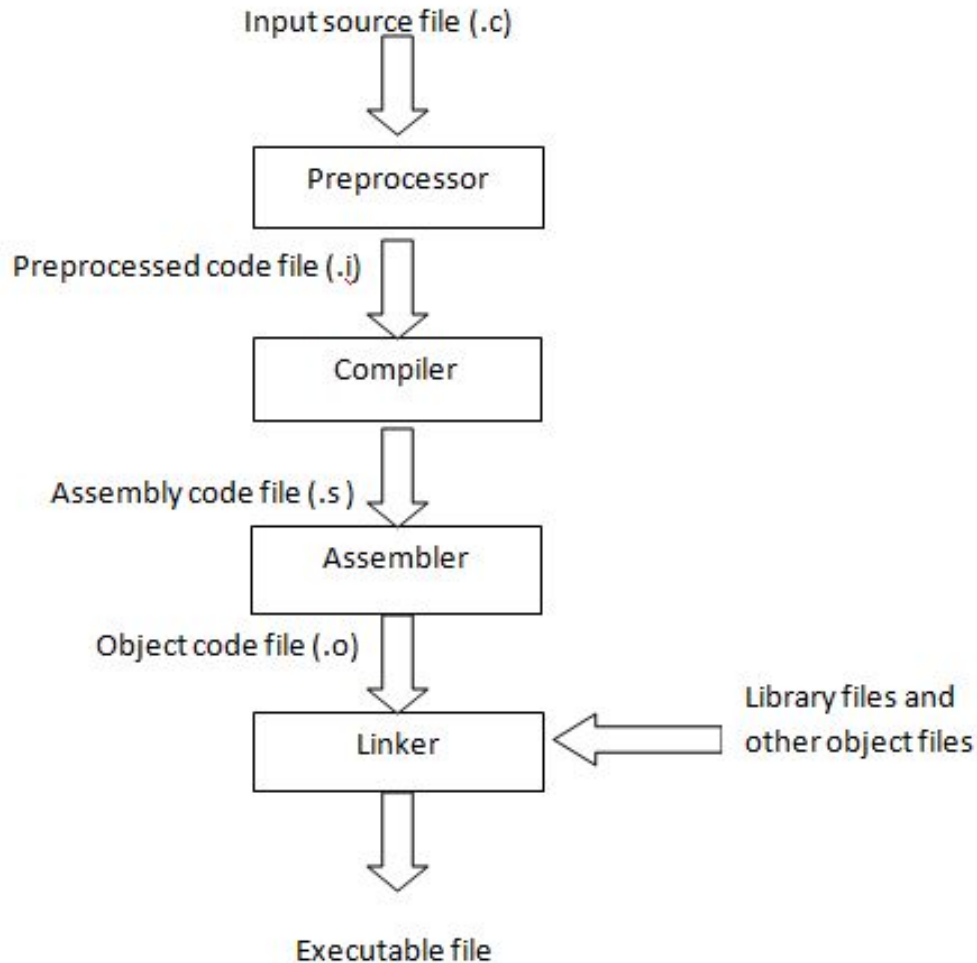
- En relación al lenguaje Assembly, 'C' **puede ser más**:
 - Confiable.
 - Escalable.
 - Portable entre distintas plataformas.
 - Mantenible.
 - Productivo.
- La eficiencia más alta se puede lograr en Assembly.
Si la velocidad de ejecución de cierta porción de código es crítica, la misma puede realizarse en Assembly.

Características generales

- 'C' es un lenguaje de programación de alto nivel de propósitos generales:
 - Imperativo.
 - Compilado.
 - Estructurado.
 - Débilmente tipado.
- El primer estándar, C89 contenía sólo 32 palabras reservadas.

Estándares de C

- **C89/C90:** ANSI C, estandarizado en 1989 por ANSI y en 1990 por ISO como ISO/IEC 9899:1990.
- **C99:** ISO/IEC 9899:1999, introdujo nuevas características como tipos de datos enteros exactos, nuevas funciones de librería estándar y el tipo de dato `bool`.
- **C11:** ISO/IEC 9899:2011, añadió soporte para multithreading, mejoras en la seguridad y nuevas funciones de librería estándar.
- **C17:** ISO/IEC 9899:2018, principalmente un estándar de corrección de errores sobre C11.
- **C23:** ISO/IEC 9899:2023, la versión más reciente, incorpora nuevas características y mejoras en la eficiencia y seguridad del código.



Preprocesamiento: El preprocesador (`cpp`) procesa directivas (`#include`, `#define`, etc.) y genera un archivo de código fuente expandido.

Compilación: El compilador (`cc`) convierte el código fuente expandido en código objeto, traduciendo el código C a código máquina.

Ensamblado: El ensamblador (`as`) convierte el código objeto en un archivo binario con código máquina.

Enlazado: El enlazador (`ld`) combina el código objeto con librerías y otros módulos, creando un ejecutable final.

Etapas de compilación

1. Lexer (análisis léxico): convierte el código fuente en **tokens**, las unidades básicas del lenguaje
2. Parser (análisis sintáctico): construye un **árbol de sintaxis abstracta** (AST) a partir de los tokens.
3. Semantic analyzer: verifica la **coherencia lógica** del AST (tipos de datos, declaración de variables, etc)
4. Optimizer: **mejora el código** intermedio generado durante la compilación para aumentar la eficiencia (eliminación de código muerto, simplificación de expresiones, etc.). Independiente del procesador.
5. Code generator: crea **código assembly**, asigna variables a registros.
6. Low-Level Optimizer: parte es específica del procesador.
7. Assembler: crea **código objeto** (código de máquina).
8. Linker/Loader: crea el **binario ejecutable**.

Directivas para el preprocesador

Empiezan con # y terminan sin ; por ejemplo:

- #define NOMBRE [valor]: macro que puede sustituir texto en el código.
- #undef NOMBRE: elimina una macro definida.
- #include <archivo.h> o “archivo.h”: incluye el contenido de un archivo
- #if, #ifdef, #ifndef, #else, #elif, #endif: controlan la **compilación condicional**.
- #pragma: métodos para proveer **instrucciones adicionales al compilador**, más allá de lo establecido por el lenguaje.

([documentación GCC](#))

Directivas para el preprocesador - ejemplos

```
// Si DEBUG_ENABLE está definido, define DEBUGOUT como printf
#ifdef DEBUG_ENABLE
#define DEBUGOUT(...) printf(__VA_ARGS__)
#else
// Si DEBUG_ENABLE no está definido, define DEBUGOUT como una macro vacía
#define DEBUGOUT(...)
#endif /* defined(DEBUG_ENABLE) */
```

```
#pragma once
```

```
// Contenido del archivo de encabezado
```

```
#ifndef HEADER_FILE_NAME_H
#define HEADER_FILE_NAME_H
```

```
// Contenido del archivo de encabezado
```

```
#endif // HEADER_FILE_NAME_H
```

Control de flujo

- **if - else:** condición lógica
- **do - while:** condición lógica
- **while:** condición lógica
- **for:** inicialización, test, reinicialización (opcionales)
- **break y continue:** salir de un loop o saltar a la reinicialización
- **switch - case:** se elige un caso basado en el valor de una expresión entera.
- **goto...**

Control de flujo - goto

Es una sentencia **no-estructurada**, que permite saltar directa e incondicionalmente hacia una etiqueta definida.

Se suele decir que “está prohibido su uso en lenguajes estructurados por las reglas de buenas prácticas.”



Tipos de dato

- La cantidad de bits asignada a un tipo de dato depende del compilador y de la arquitectura.
- Enteros:
 - char
 - short int
 - int
 - long int
- Reales:
 - float
 - double

`sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long)`

Tipos de dato estándar

- **#include<stdint.h>**
 - Tipos de dato independientes de la arquitectura.
 - uint8_t, uint16_t, uint32_t, uint64_t (sin signo).
 - int8_t, int16_t, int32_t, int64_t (con signo).
- **#include<stdbool.h>**
 - Tipo de dato booleano
 - bool
- Se logra portabilidad entre diferentes compiladores de C para diferentes arquitecturas de microcontroladores.

Operadores en C

- Aritméticos: +, -, *, /, %, ++, --
- Asignación: =
- Lógicos de comparación: <, <=, >, >=, !=, ==, &&, ||, !
- Lógicos de bits: <<, <<=, >>, >>=, &, &=, |, |=, ^, ^=, ~
- Tamaño de:
 - sizeof var
 - sizeof(tipo)

Cast

Operación que permite **cambiar temporalmente el tipo** de dato.

```
#include <stdio.h>
```

```
int main() {
```

```
    int sum = 17, count = 5;
```

```
    double mean;
```

```
    mean = (double) sum / count;
```

```
    printf("Value of mean con cast: %f\n", mean );
```

```
    mean = sum / count;
```

```
    printf("Value of mean sin cast: %f\n", mean );
```

```
    return 0;
```

```
}
```

[Compilador online para probarlo](#)

Buenas Prácticas

- Es imposible escribir siempre código que esté 100% libre de errores. Sí es posible **reducir la probabilidad de *bugs*** con un enfoque sistemático.
- Adoptar un código estilístico y respetarlo.
- Existen numerosas *coding guidelines* disponibles en la web que pueden ayudarnos a mejorar la calidad de nuestro código.
- Documentar inteligentemente el código: documentar el por qué y no el cómo [10 tips for commenting C code - EDN](#)

Buenas Prácticas en Programación en C

Claridad y Legibilidad del Código

- Utilizar nombres descriptivos para variables, funciones y constantes.
- Escribir comentarios claros y útiles.
- Mantener un formato de código consistente (indentación, espaciado, etc.).

Modularidad

- Dividir el código en funciones pequeñas y manejables.
- Utilizar archivos de encabezado (`.h`) para declarar funciones y estructuras.

Evitar Constructos Complejos

- Evitar el uso excesivo de macros y operadores complicados.
- Minimizar el uso de punteros, especialmente punteros a punteros.

Buenas Prácticas en Programación en C

Control de Errores

- Comprobar los valores de retorno de funciones críticas.
- Implementar y manejar correctamente los casos de error.

Uso Seguro de Memoria

- Evitar fugas de memoria gestionando correctamente la asignación y liberación de memoria.
- Utilizar variables locales en lugar de variables globales cuando sea posible.

Uso de Tipos de Datos Correctos

- Usar tipos de datos apropiados para el propósito específico (por ejemplo, `uint8_t` para valores que nunca serán negativos).
- Evitar conversiones implícitas de tipos.

Conformidad con Estándares

- Seguir una guía de estilo de codificación específica (por ejemplo, MISRA C, Barr Group...).

Guía de Estilo de Codificación de MISRA C

- **Descripción:** MISRA C es un conjunto de directrices de codificación para el lenguaje de programación C desarrollado por la Motor Industry Software Reliability Association (MISRA).
- **Enlace:** [MISRA Compliance:2020](#) (Propietario)
- **Características:**
 - Fomenta el uso de prácticas de codificación seguras.
 - Evita características del lenguaje que pueden llevar a errores.
 - Está orientada a aplicaciones críticas en términos de seguridad.

Estilo de Codificación de Embedded C de Barr Group

Descripción: Una guía práctica de estilo de codificación para C desarrollada por Barr Group, enfocada en la programación de sistemas embebidos.

Enlace: [Embedded C Coding Standard](#) (Libre)

Características:

- Promueve la claridad y la mantenibilidad del código.
- Incluye reglas sobre el uso de variables, funciones y estructuras de control.
- Fomenta el uso de comentarios detallados y significativos.

Guía de Estilo de Codificación de GNU C

Descripción: Una guía de estilo de codificación utilizada en el desarrollo de software para el proyecto GNU.

Enlace: [GNU Coding Standards](#) (Libre)

Características:

- Cubre aspectos generales de la programación en C.
- Enfatiza la consistencia y legibilidad del código.
- Incluye recomendaciones sobre nombres de variables, formato de código y documentación.

Guía de Estilo de Codificación de NASA

- **Descripción:** Directrices de codificación de la NASA para el desarrollo de sw en C.
- **Enlace:** [NASA C Style Guide](#) (Libre)
- **Características:**
 - Orientada a la fiabilidad y seguridad del software.
 - Contiene reglas específicas para la programación en sistemas embebidos.
 - Enfatiza la revisión y prueba del código.

Guía de Estilo de Codificación de SEI CERT C

- **Descripción:** Directrices de codificación para C desarrolladas por el Software Engineering Institute (SEI) de la Carnegie Mellon University.
- **Enlace:** [CERT C Coding Standard](#) (Libre)
- **Características:**
 - Fomenta la seguridad y la fiabilidad del software.
 - Contiene reglas para evitar vulnerabilidades comunes en C.
 - Promueve el uso de prácticas seguras de programación.

Resumen de buenas prácticas

1. **Conformidad con el lenguaje:**

Escribir código de acuerdo con las especificaciones del lenguaje

2. **Predictibilidad en la ejecución:**

Capacidad de prever y comprender cómo se ejecutará el código

3. **Programación defensiva:**

Escribir código que sea resistente a condiciones inesperadas o errores

4. **Claridad en la codificación:**

Legibilidad y comprensión del código

Conformidad con el lenguaje

1. Comportamiento No Especificado (Unspecified Behavior)

Ocurre cuando el estándar permite múltiples comportamientos posibles y no requiere que se documente cuál se elige.

```
int x;  
int y = x; // El valor de y no está especificado.
```

2. Comportamiento Dependiente de la Implementación (Implementation-Defined Behavior)

El estándar permite múltiples comportamientos, pero requiere que cada implementación documente cuál elige.

```
printf("Character: %c\n", 128); // Depende de cómo se manejen los caracteres
```

3. Comportamiento No Definido (Undefined Behavior)

Ocurre cuando el estándar no impone restricciones en lo que puede suceder. Puede provocar resultados impredecibles.

```
int arr[10];  
arr[10] = 5; // Acceso fuera de límites, comportamiento no definido.
```

Power of 10 rules ([link](#))

1. Evitar control de flujo complejo como **goto** y **recursiones**.
2. Todos los bucles deben tener límites fijos.
3. Evitar el uso de asignación dinámica de memoria.
4. Restringir las funciones a una página impresa simple.
5. Usar un mínimo de dos chequeos en tiempo de ejecución por función (assertions).
En general: pre y post condiciones.
6. Restringir la visibilidad de la información al mínimo posible.
7. Chequear el valor de retorno de todas las funciones *non-void*.
8. Usar el preprocesador moderadamente.
9. Limitar el uso de punteros a una sola desreferencia.
No usar punteros a funciones.
10. Compilar con todos los warnings activos.

Números “mágicos” o “hardcodeados”

- Aparecen en el código “sacados de la galera”

```
if (dia_actual <= 4) {  
    printf ("es laborable");  
}
```

Evitar su uso con macros variables tipo const o enumeraciones:

```
#define viernes 4
```

```
const uint8_t viernes=4;
```

```
typedef enum {lunes, martes, miercoles, jueves, viernes} dia_t
```

Constantes

- Literales y simbólicas. No pueden cambiar su valor en tiempo de ejecución.
 - `#define PI 3.1416`
 - `const float PI = 3.1416;`
- Prefijos y sufijos:
 - `0`, `0b`, `0x`, por ejemplo: `0xFF`
 - `U`, `L`, `UL`, `F`, por ejemplo: `800000UL`
- Caracteres especiales:
 - `'\n'`, `'\r'`, `'\t'`, `'\b'`, etc

Constantes - enumeración

C permite asignar valores enteros consecutivos a etiquetas mediante la palabra reservada ***enum***.

```
enum dias {LUNES, MARTES, MIERCOLES, JUEVES, VIERNES,  
SABADO, DOMINGO } ;
```

```
enum dias dia_actual;  
...  
if (dia_actual <= VIERNES) {  
    printf ("es laborable");  
}
```

Arreglos

- Una colección de datos del mismo tipo.
- Referenciados mediante un mismo nombre.
- Almacenados en posiciones de memoria físicamente contiguas.

El formato general para la declaración de una variable de tipo vector es el siguiente:

```
tipo_de_datos nombre_arreglo [tamaño];
```

```
int edades[3] = {21, 25, 27};
```


Estructuras

- Una colección de datos de distinto tipo.
- Referenciados mediante un mismo nombre.
- Se definen usando la palabra reservada ***struct***.

```
struct nombre_estructura {  
    tipo nombre_campo_1;  
    . . .  
    tipo nombre_campo_N;  
};
```

Estructuras - ejemplo

```
struct cliente {  
    char nombre[20];  
    long int dni;  
    float saldo;  
}  
  
struct cliente nuevo_cliente, clientes_vip[100];  
  
nuevo_cliente.saldo=0;  
clientes_vip[10].nombre[0] = 'P';
```

Typedef

C permite definir nuevos tipos de datos:

```
typedef tipo_de_datos nombre_nuevo_tipo;
```

Especialmente útil con enumeraciones y estructuras:

```
typedef struct {  
    char name[20];  
    char age;  
    int room_number;  
} student_t;
```

```
student_t alumnos[100]; // declaración
```

```
typedef enum {  
    LUNES, MARTES,  
    MIERCOLES, JUEVES,  
    VIERNES } dias_t;
```

```
dia_t dia_actual; // declaración
```

Funciones

- Permiten la reutilización de código.
- Es mejor que hagan 1 sólo tarea y bien.
- Estructura:

```
tipo nombre_funcion (tipo param1, ...) {  
    tipo variable_local;  
    secuencia_de_sentencias;  
    ...  
}
```

Funciones - prototipos

- Declaración que advierte al compilador sobre el **tipo, cantidad y el orden de los parámetros** que recibe una función.
- Especifica el tipo de valor de retorno de la función.
- Permite reducir el tiempo de depuración o *debug*.
- Deben ubicarse en un archivo de cabecera (*header*) .h
- Permiten utilizar funciones definidas en otro archivo fuente.
- A diferencia de la definición de la función, **terminan en ;**

Punteros

Un puntero es una variable que contiene como valor una dirección de memoria.

- Operadores unarios: * (indirección) y & (referencia)
 - & permite obtener la dirección de memoria de una variable.
 - * permite obtener el contenido de un objeto apuntado por un puntero.
 - declaración:

```
tipo_de_datos * nombre_variable_puntero;
```

Punteros

- Es recomendable inicializar los punteros con NULL.
- Chequear que el puntero tenga una dirección válida antes de usarlo.



Punteros - Ejemplo

```
int var;
```

```
int* p = NULL;
```

```
p = &var; // p apunta a var
```

```
*p = 10;    // lo apuntado por p = 10  
            // equivalente a var = 10;
```



Aritmética de punteros

```
int vector[100];    /* un vector de enteros */
int *p;              /* un puntero a enteros */
...
p = &vector[0];     /* estas dos líneas son equiv. */
p = vector

*p = 33;             /* igual que vector[0] = 33 */
*(p+1) = 44;         /* igual que vector[1] = 44 */
*(p+2) = 90;         /* igual que vector[2] = 90 */
```

Punteros y funciones

Pasaje de parámetros por referencia:

- Permite que la función modifique los valores de los parámetros.
- Permite que la función pueda devolver más de un valor.
- Optimiza el uso de la memoria.

```
void swap(int* p1, int* p2); // prototipo
int valor1, valor2;          // declaración de enteros
...
swap(&valor1, &valor2);      // llamada a la función
```

Punteros y funciones

```
void swap (int* ptr1, int* ptr2) {  
  
    int temp; // declaración de variable auxiliar.  
  
    temp = *ptr1; //guardo lo apuntado por ptr1  
  
    *ptr1 = *ptr2;    //guardo lo apuntado por ptr2.  
    *ptr2 = temp;  
}
```